

AWS DEVOPS INTERVIEW GUIDE

Scenario 1: EC2 Instance in a Public Subnet Cannot Be Accessed from the Internet

Interview Question

An EC2 instance is launched in a public subnet, but you cannot SSH into it or access the application. How would you troubleshoot it?

Answer

If an EC2 instance in a public subnet is not accessible, I verify the network configuration step by step.

Step 1: Check Public IP

Ensure the EC2 instance has a **Public IP** or **Elastic IP**.

Without a public IP, internet access is not possible.

Step 2: Verify Internet Gateway

Check that:

- An **Internet Gateway (IGW)** is attached to the VPC.
- The subnet's Route Table has:

```
0.0.0.0/0 → Internet Gateway
```

Step 3: Check Security Group

Ensure inbound rules allow the required ports.

Example:

- SSH → Port 22
- HTTP → Port 80
- HTTPS → Port 443

Step 4: Check Network ACL

Verify the NACL allows:

- Inbound traffic
- Outbound traffic
- Ephemeral ports (1024–65535)

Step 5: Verify OS Firewall

Check if the instance firewall (iptables/firewalld/UFW) is blocking traffic.

Real-Time Scenario

A production web server was unreachable even though it was running. Investigation showed that the **Route Table had no default route to the Internet Gateway**. After adding the route (0.0.0.0/0 → IGW), SSH and HTTP access were restored.

Best Practices

- Use public subnets only for internet-facing resources.
- Restrict SSH access to trusted IPs.
- Use Security Groups as the primary firewall.
- Regularly validate Route Tables and NACLs.

Interview Summary

"If a public EC2 instance is inaccessible, I first verify the Public IP, Internet Gateway, Route Table, Security Group, Network ACL, and OS firewall. These components are the most common causes of connectivity issues in a VPC."

Scenario 2: Private EC2 Instance Cannot Access the Internet

Interview Question

Your application is running on an EC2 instance in a private subnet, but it cannot download packages or access external APIs. How would you troubleshoot it?

Answer

If a private EC2 instance cannot access the internet, I verify the outbound network configuration.

Step 1: Verify NAT Gateway

Ensure the **NAT Gateway**:

- Is deployed in a **public subnet**.
 - Is in the **Available** state.
 - Has an **Elastic IP** attached.
-

Step 2: Check Route Table

Verify the private subnet's Route Table has:

```
0.0.0.0/0 → NAT Gateway
```

Without this route, the instance cannot access the internet.

Step 3: Check Public Subnet

Ensure the NAT Gateway's public subnet has:

```
0.0.0.0/0 → Internet Gateway
```

Otherwise, the NAT Gateway itself cannot reach the internet.

Step 4: Check Security Group & NACL

Verify:

- Security Group allows outbound traffic.
 - Network ACL allows inbound and outbound traffic.
-

Step 5: Test Connectivity

```
ping 8.8.8.8
```

Then test DNS:

```
ping google.com
```

- IP works, domain fails → **DNS issue**
 - Both fail → **Network routing issue**
-

Real-Time Scenario

A private EC2 instance couldn't download application updates. The private subnet's Route Table had no route to the **NAT Gateway**. After adding the default route ($0.0.0.0/0 \rightarrow$ NAT Gateway), internet connectivity was restored.

Best Practices

- Deploy NAT Gateway in a public subnet.
 - Use one NAT Gateway per Availability Zone for high availability.
 - Monitor NAT Gateway with CloudWatch.
 - Keep application servers in private subnets.
-

Interview Summary

"If a private EC2 instance cannot access the internet, I first verify the NAT Gateway, then check the Route Table, Internet Gateway, Security Groups, and Network ACLs. Finally, I test IP and DNS connectivity to determine whether the issue is related to routing or DNS."

Scenario 3: Two EC2 Instances in Different Subnets Cannot Communicate

Interview Question

Two EC2 instances are in different subnets within the same VPC, but they cannot communicate. How would you troubleshoot it?

Answer

If two EC2 instances in the same VPC cannot communicate, I verify the network configuration and security settings.

Step 1: Verify Both Instances Are in the Same VPC

Ensure both EC2 instances belong to the **same VPC**. If they are in different VPCs, communication requires **VPC Peering** or a **Transit Gateway**.

Step 2: Check Security Groups

Ensure the Security Group allows traffic from the other instance.

Example:

- SSH → Port 22
- Application Port → 8080 (or required port)

Using the **source Security Group** is more secure than allowing the entire CIDR.

Step 3: Check Network ACL

Verify the Network ACL allows inbound and outbound traffic for the required ports.

Since NACLs are **stateless**, both inbound and outbound rules must allow the traffic.

Step 4: Verify Route Tables

If both instances are in the same VPC, the **Local Route** should exist automatically.

Example:

10.0.0.0/16 → Local

If the local route is missing or modified, communication will fail.

Step 5: Test Connectivity

From one EC2 instance:

```
ping <private-ip>
```

or

```
telnet <private-ip> 8080
```

or

```
nc -zv <private-ip> 8080
```

This helps determine whether the issue is network-related or application-related.

Real-Time Scenario

A web server couldn't connect to an application server in another subnet. The Security Group only allowed traffic from the internet and did not allow traffic from the web server's Security Group. After adding the appropriate Security Group rule, communication was restored.

Best Practices

- Use **Private IPs** for communication within a VPC.
 - Reference **Security Groups** instead of opening CIDR ranges.
 - Keep the default **Local Route** intact.
 - Use **VPC Flow Logs** for network troubleshooting.
-

Interview Summary

"If two EC2 instances in different subnets cannot communicate, I first verify they are in the same VPC. Then I check Security Groups, Network ACLs, and Route Tables. Finally, I test connectivity using the private IP to determine whether the issue is with networking or the application."

Scenario 4: VPC Peering Between Two VPCs Is Not Working

Interview Question

You have configured VPC Peering between two VPCs, but the EC2 instances cannot communicate. How would you troubleshoot it?

Answer

If VPC Peering is not working, I verify the peering connection, routing, and security configuration.

Step 1: Check VPC Peering Status

Ensure the VPC Peering connection is in the **Active** state.

If it's **Pending** or **Rejected**, communication will not work.

Step 2: Verify Route Tables

Check the Route Tables in **both VPCs**.

Example:

```
VPC A: 10.1.0.0/16 → VPC Peering Connection  
VPC B: 10.0.0.0/16 → VPC Peering Connection
```

Both VPCs must have routes pointing to each other.

Step 3: Check Security Groups

Ensure the Security Groups allow traffic from the **CIDR block of the other VPC** or the required application ports.

Step 4: Check Network ACLs

Verify that the Network ACLs allow inbound and outbound traffic between both VPCs.

Step 5: Test Connectivity

From one EC2 instance:

```
ping <private-ip>
```

or

```
nc -zv <private-ip> 8080
```

This confirms whether traffic is reaching the destination.

Real-Time Scenario

An application in **VPC A** couldn't connect to a database in **VPC B**. The VPC Peering connection was active, but the Route Table in **VPC B** didn't have a route back to **VPC A**. After adding the missing route, communication was restored.

Best Practices

- Configure routes in **both** VPCs.
 - Avoid overlapping CIDR blocks.
 - Allow only required ports in Security Groups.
 - Use **Transit Gateway** instead of multiple peerings for large environments.
-

Interview Summary

"If VPC Peering is not working, I first verify the peering connection is active. Then I check Route Tables in both VPCs, validate Security Groups and Network ACLs, and test connectivity using private IPs. In most cases, missing routes or Security Group rules are the root cause."

Scenario 5: Site-to-Site VPN Connection is Down

Interview Question

Your on-premises network suddenly cannot communicate with AWS over a Site-to-Site VPN. How would you troubleshoot it?

Answer

If a Site-to-Site VPN is down, I verify the VPN status, routing, and network configuration from both the AWS and on-premises sides.

Step 1: Check VPN Status

In the AWS Console, verify:

- VPN Connection is **Available**.
- Both VPN tunnels are **UP**.

If both tunnels are down, the issue may be with the customer gateway or internet connectivity.

Step 2: Check Customer Gateway

Verify:

- Customer Gateway public IP is correct.
 - VPN device is running.
 - IPsec configuration matches AWS.
-

Step 3: Verify Route Tables

Ensure the VPC Route Table has a route to the **Virtual Private Gateway (VGW)** or **Transit Gateway (TGW)**.

Also verify that the on-premises router has routes back to the VPC.

Step 4: Check Security Groups & NACLs

Ensure the required ports and protocols are allowed between AWS and the on-premises network.

Step 5: Test Connectivity

From an EC2 instance:

```
ping <on-prem-private-ip>
```

or

```
tracert <on-prem-private-ip>
```

This helps identify where the traffic is failing.

Real-Time Scenario

A production application lost connectivity to an on-premises database. Investigation showed one VPN tunnel was down because the customer gateway firewall had blocked **IPsec traffic** after a firewall policy update. Once the firewall rule was restored, the VPN tunnel came back up and connectivity was restored.

Best Practices

- Configure **both VPN tunnels** for high availability.
 - Monitor tunnel status using CloudWatch.
 - Regularly validate route tables after network changes.
 - Consider **AWS Direct Connect** for critical, high-bandwidth workloads.
-

Interview Summary

"If a Site-to-Site VPN is down, I first check the VPN tunnel status, then verify the customer gateway, route tables, Security Groups, and Network ACLs. Finally, I test connectivity to isolate whether the issue is with AWS, the VPN tunnel, or the on-premises network."

Scenario 6: EC2 Cannot Access S3 Using a VPC Endpoint

Interview Question

Your EC2 instance uses an S3 VPC Endpoint, but it cannot access the S3 bucket. How would you troubleshoot it?

Answer

If an EC2 instance cannot access S3 through a VPC Endpoint, I verify the endpoint configuration, routing, and permissions.

Step 1: Check the VPC Endpoint

Ensure the **Gateway VPC Endpoint** for S3 is:

- Created
 - In the **Available** state
 - Associated with the correct VPC
-

Step 2: Verify Route Table

Ensure the Route Table associated with the private subnet is linked to the **S3 VPC Endpoint**.

Without the correct route, traffic won't use the endpoint.

Step 3: Check the Endpoint Policy

Verify the VPC Endpoint Policy allows access to the required S3 bucket and actions.

Example permissions:

- `s3:GetObject`
 - `s3:ListBucket`
-

Step 4: Check IAM Role & Bucket Policy

Ensure:

- The EC2 IAM Role has S3 permissions.

- The S3 Bucket Policy allows access from the IAM role or VPC Endpoint.
-

Step 5: Test Access

```
aws s3 ls s3://my-bucket
```

If access fails, review IAM permissions, bucket policy, and endpoint configuration.

Real-Time Scenario

A private EC2 instance couldn't download files from S3 despite having an IAM role. Investigation showed that the **Route Table wasn't associated with the S3 VPC Endpoint**. After associating the correct Route Table, S3 access worked without requiring internet access.

Best Practices

- Use **Gateway VPC Endpoints** for S3 and DynamoDB.
 - Apply the principle of least privilege in IAM and endpoint policies.
 - Keep application servers in private subnets.
 - Use VPC Endpoints to avoid internet traffic and NAT Gateway costs.
-

Interview Summary

"If an EC2 instance cannot access S3 through a VPC Endpoint, I verify the endpoint status, Route Table association, endpoint policy, IAM role permissions, and S3 bucket policy. Then I test access using the AWS CLI to confirm connectivity."

Scenario 7: Internet-Facing ALB Is Not Accessible

Interview Question

Your Application Load Balancer (ALB) is internet-facing, but users cannot access the application. How would you troubleshoot it?

Answer

If an internet-facing ALB is not accessible, I check the load balancer, networking, and backend instances.

Step 1: Check ALB Status

Verify the ALB is:

- In the **Active** state.
 - Configured as **Internet-facing** (not Internal).
-

Step 2: Verify Public Subnets

Ensure the ALB is deployed in **at least two public subnets**.

Each subnet's Route Table should have:

```
0.0.0.0/0 → Internet Gateway
```

Step 3: Check Security Groups

Verify the ALB Security Group allows:

- HTTP (80)
- HTTPS (443)

Also ensure the EC2 Security Group allows traffic **from the ALB Security Group**.

Step 4: Check Target Group Health

Verify the registered EC2 instances are **Healthy**.

If they are unhealthy:

- Check the health check path.
 - Verify the application is running.
 - Ensure the correct port is configured.
-

Step 5: Test the Application

From the EC2 instance:

```
curl localhost:8080/health
```

If the health endpoint fails, the issue is with the application.

Real-Time Scenario

Users couldn't access a production application through an ALB. Investigation showed the ALB Security Group allowed **HTTPS (443)** but **HTTP (80)** was missing. After updating the Security Group, the application became accessible.

Best Practices

- Deploy ALBs across multiple Availability Zones.
 - Use HTTPS with an SSL/TLS certificate.
 - Monitor Target Group health using CloudWatch.
 - Configure proper health checks.
-

Interview Summary

"If an internet-facing ALB is not accessible, I first verify the ALB type and status, then check the public subnets, Route Tables, Security Groups, and Target Group health. Finally, I test the application on the backend instance to identify whether the issue is with the network or the application."

Scenario 8: NAT Gateway Failure in Production

Interview Question

Your private EC2 instances suddenly cannot access the internet because the NAT Gateway has failed. How would you troubleshoot and resolve the issue?

Answer

If private instances lose internet access, I first verify whether the NAT Gateway is the root cause.

Step 1: Check NAT Gateway Status

Verify the NAT Gateway is in the **Available** state.

If it's **Failed** or **Deleted**, private instances won't have outbound internet access.

Step 2: Verify Route Table

Ensure the private subnet Route Table has:

```
0.0.0.0/0 → NAT Gateway
```

If the route points to the wrong NAT Gateway, update it.

Step 3: Check Internet Gateway

Ensure the NAT Gateway is in a **public subnet** and that subnet has:

```
0.0.0.0/0 → Internet Gateway
```

Without an Internet Gateway, the NAT Gateway cannot reach the internet.

Step 4: Test Connectivity

From the private EC2 instance:

```
ping 8.8.8.8
```

or

```
curl https://google.com
```

If these fail, confirm the issue is related to outbound internet access.

Step 5: Recover the NAT Gateway

If the NAT Gateway has failed:

- Create a new NAT Gateway.
 - Associate an Elastic IP.
 - Update the private subnet Route Table to point to the new NAT Gateway.
-

Real-Time Scenario

A production deployment failed because private EC2 instances couldn't download packages. Investigation showed the NAT Gateway had been accidentally deleted. A new NAT Gateway was created, the Route Table was updated, and internet connectivity was restored.

Best Practices

- Deploy **one NAT Gateway per Availability Zone** for high availability.
 - Monitor NAT Gateway using CloudWatch.
 - Avoid using a single NAT Gateway for critical production workloads.
 - Regularly validate Route Tables after infrastructure changes.
-

Interview Summary

"If private EC2 instances lose internet access, I first check the NAT Gateway status, verify the Route Table and Internet Gateway configuration, test outbound connectivity, and if necessary, create a new NAT Gateway and update the Route Table. For production, I recommend deploying a NAT Gateway in each Availability Zone for high availability."

Scenario 9: Availability Zone Failure in Production

Interview Question

One Availability Zone (AZ) becomes unavailable, affecting your application. How would you ensure the application remains available?

Answer

If an Availability Zone fails, the goal is to keep the application running by using resources in another AZ.

Step 1: Verify the Failure

Check whether the issue is limited to one **Availability Zone** using:

- EC2 Status Checks
 - CloudWatch
 - AWS Health Dashboard
-

Step 2: Check Multi-AZ Deployment

Ensure the application is deployed across **multiple Availability Zones**.

If one AZ fails, the other AZ should continue serving traffic.

Step 3: Verify Load Balancer

Check that the **Application Load Balancer (ALB)** is deployed in multiple AZs.

The ALB automatically routes traffic only to healthy instances.

Step 4: Check Auto Scaling Group

Ensure the **Auto Scaling Group** spans multiple AZs.

If instances in one AZ fail, ASG launches new instances in the healthy AZ.

Step 5: Verify Database

Ensure the database is configured for **Multi-AZ** or has a standby replica.

This prevents database downtime during an AZ failure.

Real-Time Scenario

An Availability Zone experienced a network outage, causing several EC2 instances to become unavailable. Since the application was deployed across **two AZs** with an **ALB** and **Auto Scaling Group**, traffic was automatically routed to healthy instances in the second AZ, and users experienced minimal downtime.

Best Practices

- Deploy resources across at least **two Availability Zones**.
 - Use **ALB** with cross-zone load balancing.
 - Configure **Auto Scaling** across multiple AZs.
 - Enable **Multi-AZ** for databases.
-

Interview Summary

"If an Availability Zone fails, I verify the outage, ensure the ALB and Auto Scaling Group are deployed across multiple AZs, and confirm the database is configured for Multi-AZ. This allows traffic to be redirected automatically, minimizing downtime and maintaining application availability."

Scenario 10: Design a Highly Available and Secure VPC for a Production Application

Interview Question

How would you design a highly available, secure, and scalable VPC architecture for a production application?

Answer

For a production environment, I design the VPC following AWS best practices for **high availability, security, and scalability**.

Step 1: Create the VPC

- Create a VPC with a suitable CIDR block (e.g., 10.0.0.0/16).
 - Deploy resources across **at least two Availability Zones**.
-

Step 2: Design Public and Private Subnets

- **Public Subnets:** ALB, NAT Gateway, Bastion Host (if required).
- **Private Subnets:** Application servers (EC2/EKS).
- **Private Database Subnets:** RDS databases.

This keeps critical resources isolated from the internet.

Step 3: Configure Internet Access

- Attach an **Internet Gateway (IGW)** to the VPC.
 - Deploy a **NAT Gateway** in each Availability Zone for private subnet internet access.
-

Step 4: Configure Security

- Use **Security Groups** with least-privilege access.
 - Use **Network ACLs** for subnet-level security.
 - Store secrets in **AWS Secrets Manager** or **SSM Parameter Store**.
-

Step 5: Ensure High Availability

- Deploy the **ALB** across multiple AZs.
 - Configure the **Auto Scaling Group** across multiple AZs.
 - Use **RDS Multi-AZ** for the database.
-

Step 6: Monitoring & Logging

Enable:

- CloudWatch
- VPC Flow Logs
- CloudTrail

These help with monitoring, troubleshooting, and auditing.

Real-Time Scenario

For an e-commerce application, we designed a VPC with **public subnets for the ALB and NAT Gateways**, **private subnets for application servers**, and **private database subnets for RDS** across two Availability Zones. During an AZ outage, the ALB redirected traffic to healthy instances, and the Multi-AZ database failed over automatically, ensuring minimal downtime.

Best Practices

- Use public subnets only for internet-facing resources.
 - Keep application and database servers in private subnets.
 - Deploy across multiple AZs.
 - Enable VPC Flow Logs, CloudTrail, and CloudWatch.
 - Follow the principle of least privilege for Security Groups and IAM.
-

Interview Summary

"For a production VPC, I use public subnets for the ALB and NAT Gateways, private subnets for application servers, and isolated database subnets for RDS. I deploy resources across multiple Availability Zones, use Auto Scaling and ALB for high availability, secure the environment with Security Groups and NACLs, and enable monitoring with CloudWatch, CloudTrail, and VPC Flow Logs."

Scenario 11: VPC Flow Logs Show Dropped Traffic

Interview Question

Users report intermittent connectivity issues. How would you use VPC Flow Logs to troubleshoot?

Answer

When network traffic is failing, I enable or review **VPC Flow Logs** to determine whether traffic is being accepted or rejected.

Step 1: Check VPC Flow Logs

Review Flow Logs in CloudWatch or S3.

Look for:

- ACCEPT
 - REJECT
-

Step 2: Identify the Source and Destination

Verify:

- Source IP
- Destination IP
- Port
- Protocol

This helps identify where traffic is failing.

Step 3: Check Security Groups & NACLs

If Flow Logs show **REJECT**, verify:

- Security Group rules
 - Network ACL rules
-

Step 4: Fix the Issue

Update the required Security Group or NACL rule and test connectivity.

Real-Time Scenario

An application couldn't connect to an RDS database. VPC Flow Logs showed **REJECT** entries on port **3306**. A Network ACL rule was blocking the traffic. After updating the NACL, connectivity was restored.

Interview Summary

"I use VPC Flow Logs to determine whether traffic is accepted or rejected, identify the affected ports and IPs, and then verify Security Groups and Network ACLs to resolve the issue."

Scenario 12: DNS Resolution Not Working Inside the VPC

Interview Question

Your EC2 instance can reach other servers using IP addresses, but domain names are not resolving. How would you troubleshoot it?

Answer

If DNS resolution fails, I check the VPC DNS settings, DHCP options, and the instance configuration.

Step 1: Test DNS

Check whether the issue is DNS or network connectivity.

```
ping 8.8.8.8
```

Then:

```
ping google.com
```

- **IP works, domain fails** → DNS issue.

- **Both fail** → Network issue.
-

Step 2: Check VPC DNS Settings

Verify the VPC has:

- **DNS Resolution = Enabled**
 - **DNS Hostnames = Enabled**
-

Step 3: Check DHCP Options Set

Ensure the VPC is using the correct DNS server (usually **AmazonProvidedDNS**).

Step 4: Check Instance DNS Configuration

Verify the DNS server configured on the instance.

```
cat /etc/resolv.conf
```

Step 5: Restart Network Service (if required)

If DNS settings were updated, restart the network service or reboot the instance if necessary.

Real-Time Scenario

A private EC2 instance couldn't connect to an external API using its domain name, but IP connectivity worked. Investigation showed the VPC had **DNS Resolution disabled**. After enabling it, domain name resolution worked normally.

Best Practices

- Keep **DNS Resolution** and **DNS Hostnames** enabled.
 - Use **AmazonProvidedDNS** unless custom DNS is required.
 - Regularly validate DHCP option sets after changes.
-

Interview Summary

"If DNS resolution fails, I first determine whether it's a DNS or network issue by testing IP and domain connectivity. Then I verify VPC DNS settings, DHCP options, and the instance's DNS configuration. Finally, I retest to confirm that name resolution is working."

Scenario 13: Interface VPC Endpoint Is Not Working

Interview Question

Your EC2 instance in a private subnet cannot access AWS services such as Secrets Manager, Systems Manager, or ECR through an Interface VPC Endpoint. How would you troubleshoot it?

Answer

If an Interface VPC Endpoint is not working, I verify the endpoint configuration, DNS, Security Groups, and IAM permissions.

Step 1: Check VPC Endpoint Status

Ensure the Interface VPC Endpoint is in the **Available** state and deployed in the correct subnets.

Step 2: Verify Security Groups

Ensure the VPC Endpoint's Security Group allows inbound **HTTPS (Port 443)** from the EC2 instance.

Step 3: Check Private DNS

Verify **Private DNS** is enabled for the Interface Endpoint.

Without it, the service endpoint may resolve to the public endpoint instead of the private one.

Step 4: Verify IAM Permissions

Ensure the EC2 IAM Role has the required permissions to access the AWS service.

Example:

- `secretsmanager:GetSecretValue`
 - `ssm:GetParameter`
-

Step 5: Test Connectivity

Test access from the EC2 instance.

```
aws secretsmanager get-secret-value --secret-id my-secret
```

If it fails, review the endpoint configuration, Security Groups, and IAM permissions.

Real-Time Scenario

An application running in a private subnet couldn't retrieve secrets from **AWS Secrets Manager**. The Interface VPC Endpoint was configured correctly, but its **Security Group didn't allow inbound port 443** from the application servers. After updating the Security Group, the application successfully retrieved the secrets.

Best Practices

- Enable **Private DNS** for Interface Endpoints.
 - Restrict access using Security Groups.
 - Use Interface Endpoints for services like **SSM, Secrets Manager**, and **ECR**.
 - Monitor endpoint usage with **CloudTrail** and **VPC Flow Logs**.
-

Interview Summary

"If an Interface VPC Endpoint is not working, I verify the endpoint status, Security Group, Private DNS configuration, and IAM permissions. Then I test connectivity using the AWS CLI to ensure the service is accessible through the private endpoint."

Scenario 14: Network ACL Is Blocking Traffic

Interview Question

Your Security Group is correctly configured, but the application is still unreachable. How would you troubleshoot a Network ACL issue?

Answer

If the Security Group is correct but traffic is still blocked, I check the **Network ACL (NACL)** because it acts as a subnet-level firewall.

Step 1: Check the NACL Rules

Verify the NACL associated with the subnet.

Ensure it allows:

- Required application ports (e.g., 80, 443, 22)
 - Ephemeral ports (1024–65535)
-

Step 2: Verify Inbound and Outbound Rules

Unlike Security Groups, **NACLs are stateless**.

This means both **Inbound** and **Outbound** rules must allow the traffic.

Step 3: Check Rule Order

NACL rules are evaluated from the **lowest rule number to the highest**.

A lower-numbered **Deny** rule overrides a later **Allow** rule.

Step 4: Use VPC Flow Logs

Check VPC Flow Logs for **REJECT** entries to identify the blocked traffic.

Step 5: Test Connectivity

After updating the NACL, verify connectivity.

```
nc -zv <private-ip> 443
```

or

```
telnet <private-ip> 443
```

Real-Time Scenario

An application server couldn't connect to an RDS database even though the Security Groups were correct. Investigation using **VPC Flow Logs** showed **REJECT** entries on port **3306**. A Network ACL had an inbound **Deny** rule. After updating the NACL, database connectivity was restored.

Best Practices

- Use **Security Groups** as the primary firewall.
 - Keep NACLs simple unless subnet-level filtering is required.
 - Always allow ephemeral ports for return traffic.
 - Use VPC Flow Logs to troubleshoot blocked traffic.
-

Interview Summary

"If connectivity fails despite correct Security Groups, I check the Network ACL, verify inbound and outbound rules, ensure rule order is correct, review VPC Flow Logs for rejected traffic, and test connectivity after making the required changes."

Scenario 15: Secure Communication Between Two AWS Accounts

Interview Question

Your application in AWS Account A needs to communicate securely with an application in AWS Account B without using the internet. How would you design and troubleshoot the solution?

Answer

The solution depends on the connectivity requirement:

- **VPC Peering** → Two VPCs with simple communication.
 - **Transit Gateway** → Multiple VPCs or multiple AWS accounts.
 - **AWS PrivateLink** → Secure access to a specific application/service without exposing the entire VPC.
-

Step 1: Choose the Right Connectivity Option

- **VPC Peering**: Small environments with two VPCs.
 - **Transit Gateway**: Large environments with many VPCs.
 - **PrivateLink**: Expose only a specific service securely.
-

Step 2: Verify Routing

Ensure Route Tables contain routes to the destination VPC or Transit Gateway.

Without proper routes, traffic won't reach the destination.

Step 3: Check Security

Verify:

- Security Groups allow the required ports.
 - Network ACLs allow inbound and outbound traffic.
-

Step 4: Test Connectivity

From the source EC2 instance:

```
nc -zv <private-ip> 443
```

or

```
curl http://<private-ip>
```

This confirms whether the application is reachable.

Step 5: Monitor and Troubleshoot

Use:

- **VPC Flow Logs**
- **CloudWatch**
- **Reachability Analyzer**

to identify routing or security issues.

Real-Time Scenario

A Jenkins server in **Account A** needed to access an internal deployment API hosted in **Account B**. Since only one application needed to be exposed, **AWS PrivateLink** was implemented instead of VPC Peering. This allowed secure private communication without exposing the entire VPC or using the public internet.

Best Practices

- Use **VPC Peering** for simple one-to-one connectivity.
 - Use **Transit Gateway** for large multi-VPC or multi-account environments.
 - Use **PrivateLink** to expose only specific services.
 - Enable **VPC Flow Logs** and use **Reachability Analyzer** for troubleshooting.
-

Interview Summary

"For secure communication between AWS accounts, I choose VPC Peering for simple connectivity, Transit Gateway for large-scale networking, or PrivateLink when

exposing only a specific service. I then verify Route Tables, Security Groups, Network ACLs, and test connectivity using private IPs. Finally, I use VPC Flow Logs and Reachability Analyzer to troubleshoot any issues."
